

VANGUARD CLIENT TELEMETRY PROJECT

Code Documentation

Telemetry Generation | Feature Engineering | Frustration Scoring | Dashboard Visualization

Phase 1

Telemetry Ingestion

Phase 2

AI Scoring Pipeline

Phase 3

Analytics Dashboard

Prepared by

Sabrina Akter

Geri Koka

Shengyang Dong

Guanxin Zheng

Manish Gurung

Junkai Ge

Drexel University | April 2026

Table of Contents

Section	Topic	What it covers
1	Project Overview	System purpose, three-phase architecture, and data flow
2	Phase 1 - Telemetry Generation & Ingestion	Client SDK, ingestion server, and Playwright simulator
3	Phase 2 - Feature Engineering & AI Scoring	Preprocessing, model training, inference, and scheduling
4	Phase 3 - Dashboard Visualization & Analytics	Backend APIs, dashboard views, metrics, and AWS data sources
5	Quick Start Checklist	Recommended end-to-end run sequence

1. Project Overview

Vanguard Client Telemetry is a three-phase system that collects synthetic user interaction telemetry, converts behavioral signals into machine-learning features, generates frustration predictions, and displays results through an administrative analytics dashboard.

System Architecture

Phase 1	Phase 2	Phase 3
Telemetry Generation	Feature Engineering & Scoring	Dashboard Visualization
Client SDK + Node/Express ingestion + Playwright simulations	Preprocessing + Autoencoder/Isolation Forest + SageMaker/local inference	Flask APIs + React dashboard + alerts, queue, and session details

End-to-End Data Flow

```
Browser events -> /api/telemetry -> NDJSON/S3 raw logs -> preprocess.py features -> train.py /  
inference.py predictions -> S3 results -> Flask API -> React dashboard
```

Core Project Goals

- Capture client-side behavioral signals such as rage clicks, scroll depth, idle time, navigation loops, and form abandonment.
- Standardize telemetry into a consistent camelCase schema for downstream processing.
- Aggregate raw events into session-level feature vectors and generate frustration scores.
- Expose predictions and session analytics in dashboard views for monitoring, triage, and admin review.

2. Phase 1 - Telemetry Generation & Ingestion

Phase 1 establishes the telemetry foundation: a browser SDK captures behavioral events, a Node.js ingestion server stores them as NDJSON, and a Playwright runner generates synthetic user sessions for testing and model input.

Setup and Configuration

Runtime	Node.js v14+ with npm
Install packages	npm install
Install browser engine	npx playwright install chromium
Log destination	logs/telemetry_logs.ndjson
Telemetry endpoint	POST /api/telemetry
Schema standard	camelCase fields such as serverReceivedAt, sessionId, pageRoute, eventType
npm install npx playwright install chromium npm run build	

public/telemetry.js		Component
Purpose	Client-side SDK loaded by each mock website page. It monitors browser interactions and emits telemetry events to the ingestion API.	

Key features	• Captures global user interactions across page assets. • Detects rage clicks when 3+ rapid clicks target the same DOM element within 800 ms. • Tracks scroll-depth milestones at 25%, 50%, 75%, and 100%. • Detects idle sessions after 30 seconds of inactivity. • Captures form abandonment when an engaged form is left without submit. • Standardizes flow tracking with flow_start, flow_step, flow_complete, and flow_abandon.
Storage / integration	• Reads DOM events and browser localStorage for a persistent multi-page sessionId. • Writes JSON telemetry objects via HTTP POST to /api/telemetry.

Common commands

```
Telemetry.init('page_name');
Telemetry.emit('button_click', { button: 'save' });
```

server.js Component	
Purpose	Node.js Express ingestion backend that serves the mock website and persists telemetry events.
Key features	• Serves static financial simulation pages from public/. • Accepts telemetry payloads through POST /api/telemetry. • Appends events safely as newline-delimited JSON. • Adds serverReceivedAt timestamps for ordering and auditability.
Storage / integration	• Reads static assets from public/. • Writes append-only records to logs/telemetry_logs.ndjson.

Common commands

```
node server.js
# Local app: http://localhost:3000
```

run.js	Component
Purpose	Playwright-backed synthetic simulation runner that automates user sessions and drives realistic telemetry generation.
Key features	• Supports normal_user, frustrated_user, lost_user, and error_user personas. • Simulates rapid refocusing, U-turn navigation, retries, and console/network exceptions. • Routes browser events to the live server for Phase 2 model input. • Supports configurable session counts, scenario mixes, and optional output snapshots.
Storage / integration	• Reads behavioral scripts from src/ and CLI parameters. • Writes events to server.js via HTTP POST and optionally to a local JSONL file.

Common commands

```
node run.js --baseUrl http://localhost:3000 --sessions 20
node run.js --baseUrl http://localhost:3000 --sessions 10 --scenarioMix
normal:0.5,frustrated:0.3,lost:0.1,error:0.1 --output my_events.jsonl
```

Phase 1 CLI Reference

--baseUrl	Base URL of the website. Default: http://localhost:8000.
--sessions	Number of distinct browser simulation sessions. Default: 50.
--scenarioMix	Scenario probability ratios. Default: normal:0.4,frustrated:0.3,lost:0.2,error:0.1.
--output	Local .jsonl backup path for generated sessions.

Phase 1 Output Contract

Primary artifact	logs/telemetry_logs.ndjson
------------------	----------------------------

Vanguard Client Telemetry | Code Documentation

Record format	One JSON object per line for reliable downstream parsing.
Required fields	serverReceivedAt, sessionId, userId when available, pageRoute, eventType, timestamp, url, and metadata.
Validation check	Confirm that newly generated sessions appear in the NDJSON log before running Phase 2 preprocessing.

3. Phase 2 - Feature Engineering & AI Scoring

Phase 2 transforms raw telemetry into model-ready features, trains a two-model anomaly ensemble, and produces frustration predictions for dashboard consumption. The pipeline is designed for AWS S3/SageMaker integration while still supporting local testing paths.

Environment and Shared Storage

```
pip install pandas numpy scikit-learn tensorflow boto3 sagemaker schedule joblib sqlalchemy psycopg2-binary
```

Raw telemetry bucket	sagemaker-us-east-1-197337164107/raw/
Pipeline/model bucket	sagemaker-studio-i0gutcxdy
Model artifacts	frustration-model/models/
Preprocessed data	frustration-model/preprocessed_data/
Inference results	frustration-model/results/
Processed markers	frustration-model/processed_markers/
Configuration	config.py values such as FRUSTRATION_MODEL_BUCKET and AWS_REGION can be overridden if needed.

preprocess.py		Component
Purpose	Ingests raw NDJSON/JSONL telemetry and creates normalized session-level feature vectors for training and inference.	
Key features	• Reads raw telemetry locally or directly from S3. • Aggregates events into 13 key session-level metrics. • Normalizes features using Z-score or MinMax statistics. • Splits training/testing data and uploads outputs to S3. • Can write extracted features to PostgreSQL/RDS.	

Storage / integration

- Reads from local paths or the raw S3 bucket.
- Writes to frustration-model/preprocessed_data/ unless --local-only is used.

Common commands

```
python preprocess.py --input s3://sagemaker-us-east-1-197337164107/raw/data.ndjson --output ./preprocessed
python preprocess.py --input ./local_raw_data.ndjson --output ./local_features --local-only
python preprocess.py --input s3://... --output ./out --no-normalize --schema custom_schema.json
```

--input	S3 URI or local raw data path. Default: /opt/ml/processing/input.
--output	Local directory for output files. Default: /opt/ml/processing/output.
--max-events	Limit raw event count for testing.
--no-normalize	Disable feature normalization.
--normalize-method	zscore by default; minmax also supported.
--format	Output format: csv by default or npy.
--include-session-id	Include sessionId in output.
--include-timestamp	Include session start timestamp.
--include-user-id	Include userId in output.
--schema	Path to custom event mapping schema JSON.
--local-only	Skip S3 and RDS writes.

train.py

Component

Purpose

Trains and packages the project frustration-detection model ensemble.

Key features	• Trains a TensorFlow/Keras Autoencoder and Scikit-Learn Isolation Forest. • Validates artifacts before saving; unsafe all-zero predictions fail upload. • Packages .keras, .pkl, and metadata into timestamped model_vX.tar.gz artifacts. • Auto-increments model versions by checking existing S3 files. • Supports a semi-supervised feedback loop using manually labeled data.
Storage / integration	• Reads training/testing CSVs and optionally frustration-model/labelled/manually_labeled.csv. • Writes model artifacts to frustration-model/models/model_vX.tar.gz.

Common commands

```
python train.py --train ./preprocessed/train.csv --test ./preprocessed/test.csv
python train.py --train ./train.csv --test ./test.csv --epochs 200 --use-feedback
```

--train	Required path to preprocessed training CSV.
--test	Required path to preprocessed testing CSV.
--model-dir	Temporary local model directory. Default: ./temp_models.
--epochs	Autoencoder training epochs. Default: 100.
--learning-rate	Optimizer learning rate. Default: 0.001.
--n-estimators	Isolation Forest tree count. Default: 100.
--contamination	Expected anomaly proportion. Default: 0.1.
--use-feedback	Enable threshold tuning with labeled data.
--labeled-data-key	S3 key for labeled feedback CSV. Default: frustration-model/labelled/manually_labeled.csv.

inference.py

Component

Purpose	Runs batch scoring with trained model artifacts and outputs session-level prediction results.
Key features	• Loads Autoencoder, Isolation Forest, scaler, and metadata. • Accepts preprocessed session feature vectors. • Combines model outputs into frustration scores. • Maps scores into Low, Medium, and High severity labels. • Supports batch predictions for multiple sessions.
Storage / integration	• Reads models from frustration-model/models/ and features from frustration-model/preprocessed_data/. • Writes prediction outputs to frustration-model/results/.

Common commands

```
python inference.py --input ./preprocessed/features.csv --output ./inference_results
python inference.py --input s3://sagemaker-studio-i0gutcxdy/frustration-model/preprocessed_data/features.csv --output ./inference_results
```

--input	Required local path or S3 URI to the feature file.
--output	Local output directory for prediction results.
--model-path	Local model directory or model artifact path.
--local-only	Save locally and skip S3 upload.
--endpoint-name	Optional SageMaker endpoint name for endpoint-based inference.

client.py		Component
Purpose	Main resilient inference workflow that syncs the newest model, handles endpoint fallback, and prevents duplicate processing.	

Key features	• Brain Swap Sync: downloads and validates the newest model_vX.tar.gz from S3. • Routes scoring to SageMaker first, then falls back to local models if the endpoint is unavailable. • Uses .done marker files to avoid duplicate processing. • Generates audited predictions.csv with scores and Low/Medium/High severity.
Storage / integration	• Reads raw telemetry and model artifacts from S3. • Writes markers, predictions, and feature outputs back to S3.

Common commands

```
python client.py --input s3://sagemaker-us-east-1-197337164107/raw/batch1.ndjson
python client.py --input ./raw_data.ndjson --no-endpoint --local-only
python client.py --input s3://.../data.ndjson --endpoint-name my-endpoint --update-endpoint
```

--input	Required S3 URI or local raw telemetry path.
--schema	Platform schema JSON. Default: config/schema_default.json.
--local-only	Save local outputs only and skip S3 upload.
--endpoint-name	SageMaker endpoint name. Default: frustration-scoring-endpoint.
--no-endpoint	Force local Brain model scoring.
--update-endpoint	Deploy/update SageMaker endpoint to latest S3 model before scoring; requires --endpoint-name.

scheduler.py

Component

Purpose	Automation process that continuously monitors S3 and triggers the Phase 2 scoring workflow when new telemetry arrives.
Key features	• Monitors the raw telemetry S3 folder for new NDJSON/JSONL files. • Uses processed markers to prevent repeat processing. • Triggers preprocessing and inference automatically. • Uploads feature files, predictions, and marker files to S3.

Vanguard Client Telemetry | Code Documentation

Storage / integration	• Reads from raw telemetry bucket and processed marker path. • Writes to preprocessed data, inference results, and processed marker paths.
Notes	Run command: <code>python scheduler.py</code> . Expected workflow: check raw S3 files -> compare markers -> preprocess -> infer scores/severity -> upload results -> create marker file.

4. Phase 3 - Dashboard Visualization & Analytics

Phase 3 provides the administrative dashboard used to review processed telemetry sessions, frustration predictions, behavioral metrics, alerts, and queue prioritization. The dashboard pulls the latest prediction, feature, and raw-event artifacts from AWS S3 through backend APIs.

Dashboard Views

Session Overview	• Lists processed sessions from the latest prediction run. • Shows sessionId, scenario, score, severity, status, event count, and timestamp. • Includes summary statistics such as total sessions and severity distribution.
Alerts View	• Displays Medium and High severity sessions. • Helps administrators quickly identify sessions that may require attention.
Queue View	• Prioritizes sessions by status and severity. • Sorting order: ongoing sessions, High severity, Medium severity, Low severity.
Session Detail View	Shows individual session metadata, scenario, score, severity, status, event count, behavioral metrics, and outcome data.
Event Timeline	• Displays timestamped telemetry events for a selected session. • Includes event type, route, userId, and event metadata.

Metrics Displayed

Aggregated Metrics • Total Clicks • Error Count • Retry Count • Rage Click Count • Average Dwell Time	Behavior Metrics • Page View Count • Event Count • Unique Route Count • Field Change Count • Flow Success Count
--	--

- | | |
|--|---|
| <ul style="list-style-type: none"> • Navigation Loop Count • Backtrack Rate • Form Abandonment • Idle Timeout Count • Raw Event Count | <ul style="list-style-type: none"> • Flow Failure Count • Session Duration • Average Time Between Events |
|--|---|

Scenario Detection

- First priority: flowName values from telemetry metadata.
- Second priority: pageRoute values from navigation events.
- Fallback: navigation labels when no scenario information is available.
- Examples: Trade, Create Account, Holdings, Account Home, General Navigation.

AWS Data Sources

Prediction outputs	s3://sagemaker-studio-i0gutcxdy/frustration-model/results/
Feature vectors	s3://sagemaker-studio-i0gutcxdy/frustration-model/preprocessed_data/
Raw telemetry events	s3://sagemaker-us-east-1-197337164107/raw/

Backend Setup

```
aws configure
# Default Region: us-east-1
# Default Output Format: json

python3 -m venv .venv
source .venv/bin/activate
pip install flask flask-cors boto3 pandas
python server.py
# Default API endpoint: http://localhost:4000
```

Frontend Setup

```
npm install
```

```
npm run dev  
# Default dashboard URL: http://localhost:5173
```

Main Dashboard Files

File	Description
server.py	Primary backend API service used by the dashboard.
MetricsGrid.jsx	Displays aggregated session metrics.
SessionDetail.jsx	Displays detailed session information and analytics.
Sessions.jsx	Displays processed sessions from the latest prediction run.
Alerts.jsx	Displays Medium and High severity sessions.
Queue.jsx	Displays prioritized sessions based on severity and status.

Notes

- The dashboard dynamically loads the latest prediction and feature files generated by Phase 2.
- No manual dashboard updates are required when new prediction outputs are generated.
- Refreshing the dashboard displays the most recent processed sessions and analytics available in S3.

5. Quick Start Checklist

Use this checklist when demonstrating the full Vanguard Client Telemetry workflow from telemetry generation to dashboard visualization.

Step	Action	Purpose
1	Start Phase 1 ingestion	Install dependencies and run server.js to serve the mock dashboard and receive telemetry.
2	Generate synthetic sessions	Run run.js with the desired session count and scenario mix.
3	Preprocess telemetry	Run preprocess.py to produce normalized session-level feature vectors.
4	Train or sync model artifacts	Use train.py for model generation or client.py Brain Swap to pull the latest model.
5	Run inference	Generate predictions.csv with frustration scores and severity labels.
6	Start backend API	Configure AWS credentials and run the Flask backend at http://localhost:4000.
7	Launch dashboard	Run the React development server and review sessions at http://localhost:5173.

Recommended Demonstration Command Flow

```
node server.js
node run.js --baseUrl http://localhost:3000 --sessions 20
python preprocess.py --input s3://sagemaker-us-east-1-197337164107/raw/data.ndjson --output
./preprocessed
python client.py --input s3://sagemaker-us-east-1-197337164107/raw/batch1.ndjson
```

```
python server.py  
npm run dev
```